Welcome

let's get started

By Swarnima Mitra

Function Call

The syntax of calling a function that does not accept parameter is simply name of the function followed by parenthesis,

function_name()

Function call statement has the following syntax when it accepts parameters:

function_name(var1, var2, var3.....)

Function Parameters

A function can take param which are some values that are passed to it so that the fn can **manipulate** them to produce the desired result.

Key points to remember while calling the function

- The fn name, number, and type of arguments in the call must match the definition.
- Passing more or fewer parameters than expected, results in an error
- Indented statements form the function body.

Program to demonstrate the mismatch between function parameters

```
def func(i, j):
        print (“Hello World”, i, j)
func(5)
```

OUTPUT:

TypeError: func() takes exactly 2 arguments (1 given)

The indented statements form body of the function

Program to demonstrate the mismatch between function parameters and arguments

```
def func(i):
        print (“Hello World”, i)
func(5, 5)
```

OUTPUT:

TypeError: func() takes exactly 1 arguments (2 given)

Program to demonstrate the mismatch between function parameters and arguments

```
def func(i):    # fun definition header accepts a var with name i
        print("Hello World", i)
j = 10
func(j)    # Fn is called using variable j
```

OUTPUT:

Hello World 10

Program to demonstrate the mismatch between function parameters and arguments

```
def func(i):
        print ("Hello World" +  i)
func(5)
```

OUTPUT:

TypeError: cannot concatenate 'str' and 'int' objects

Program to demonstrate that the arguments may be passed in the form of expressions to the called function.

```
def func(i):
        print ("Hello World",i)
func(5+2*3)
```

OUTPUT:

Hello World 11

Before calling a function, you must define it.

Program to add two integers from user using function that accepts two arguments

```
def total(a,b):   #fn accepting parameters
        result = a+b
        print(“”Sum of”,a, “and”, b, “=”, result)
a = int(input(“Enter 1st number: ”))
b = int(input(“Enter 2nd number: ”))
total(a,b)          #function call with 2 arguments
```

OUTPUT:

Enter the first number : 10

Enter the first number : 20

Sum of 10 and 20 : 30

It is a logic error if the arguments in the function call are placed in a wrong order

Parameter Passing: Mutable vs Immutable

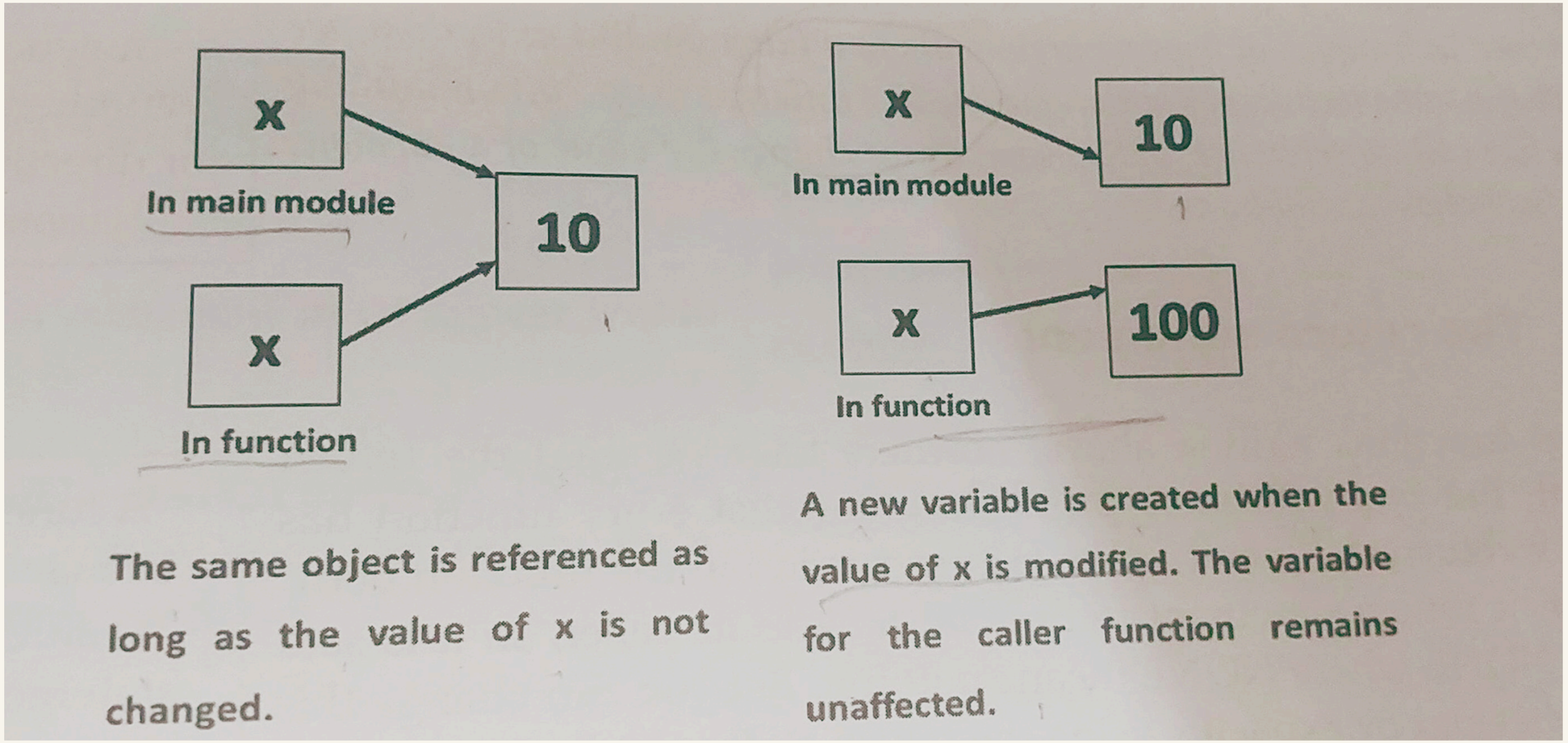
- Python uses "Call-by-Object" or "Call by Object Reference."

- Immutable Objects (e.g., integers, strings, tuples):
  - Passed using a call-by-value technique.
  - The object reference is passed, but its value remains unchanged in the function.

- Mutable Objects (e.g., lists, dictionaries):
  - Passed by object reference.
  - Their contents can be modified within the function.

Key Behavior:

- If an immutable object is reassigned inside a function, Python creates a new local variable.
- The original object in the caller function remains unaffected.
- Mutable objects, when modified inside a function, reflect changes in the caller function.



- If an integer x = 10 is passed to a function, it remains unchanged unless reassigned.
- If reassigned (e.g., x = 100), Python creates a new local variable, leaving the original unaffected.

```
# Call by Value (Immutable Example)
def modify_immutable(x):
    x = 100  # Reassigning creates a new local variable
    print("Inside function (Immutable):", x)

num = 10
modify_immutable(num)
print("Outside function (Immutable):", num)  # Original value remains unchanged

# Call by Reference (Mutable Example)
def modify_mutable(lst):
    lst.append(4)  # Modifying the original list
    print("Inside function (Mutable):", lst)

my_list = [1, 2, 3]
modify_mutable(my_list)
print("Outside function (Mutable):", my_list)  # Original list is modified
```

OUTPUT

Inside function (Immutable): 100
Outside function (Immutable): 10

Inside function (Mutable): [1, 2, 3, 4]
Outside function (Mutable): [1, 2, 3, 4]

Call by Value (Immutable - Integers)
Call by Reference (Mutable - Lists)

Write a program that finds greater of three given numbers using functions .Pass the numbers as arguments.

```
def _greatest(a, b, c):
    """ Function to find the greatest of three numbers"""
    if a >= b and a >= c:
        return a
    elif b >= a and b >= c:
        return b
    else:
        return c

# Taking input from the user
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))
num3 = int(input("Enter third number: "))

# Calling the function and displaying the result
greatest = _greatest(num1, num2, num3)
print(f"The greatest number is: {greatest}")
```

WAP that prints the time taken to execute a program in Python

```
import time

def sample_task():
    """Function to simulate some task"""
    total = 0
    for i in range(1, 1000000):
        total += i  # Simple loop to add numbers
    return total

# Start time
start_time = time.time()

# Execute the function
result = sample_task()

# End time
end_time = time.time()

# Calculate execution time
execution_time = end_time - start_time

print(f"Execution Time: {execution_time:.6f} seconds")
```

Output:

Execution Time: 0.025364 seconds

TASK

1.Write a function that returns the absolute value of a number

2. Write a function is_prime() that returns a 1 if the argument passed to it is a prime number and a 0 otherwise

The return statement

- **Implicit Return:** If a function does not explicitly return a value, Python automatically returns None at the end of the function.
- **Explicit Return:** You can override this default behavior by using the return statement to return a specific value to the caller.
- **Return with No Arguments:** A return statement without any arguments is equivalent to return **None.**

- Purpose of the return Statement: It serves two main functions—returning a value to the caller and exiting the function, transferring control back to the caller.
- Optional Expression: The expression in the return statement is optional. If no expression is provided, the function returns None by default.

retrun [expression]

```python
def greet(name):
    print("Hello, " + name + "!")

greet("Python")   # Function call 1
x = greet("World")  # Function call 2 and assigning the return value to x
print(x)        # Printing the value of x
print(greet("Again"))  # Function call 3 inside print
```

OUTPUT:

```
Hello, Python!
Hello, World!
None
Hello, Again!
None
```

```python
def greet(name):
    return "Hello, " + name + "!"

print(greet("Python"))  # This will print the returned string
x = greet("World")  # Assigning the return value to x
print(x)  # Printing x, which holds the returned string
print(greet("Again"))  # Calling greet inside print
```

OUTPUT:

```
Hello, Python!
Hello, World!
Hello, Again!
```

```
# To write another function , whihc return an integgerto the caller

def cube(x):
   return(x*x*x)
num= 10
result = cube(num)
print('Cube of ', num, ' = ', result)
```

OUTPUT:

Cube of 10 = 1000

- The function prints messages but does not return anything explicitly.
- Without a return statement, Python functions return None by default.

```
#Program to demonstrate Keyword Argument

def display(str, int_x, float_y):
    print("The string is : ", str)
    print("The integer value is : ", int_x)
    print("The float value is : ", float_y)
display(float_y =1.23344, str = "Hello",int_x= 312 )
```

What is the output when display function is called with keyword arguments for salary, name and age?

```
def display(name, age, salary):
    print("Name is : ", name)
    print("The age is : ", age)
    print("The salary amount is: ",salary)
n = "Aadi"
a = 22
s = 123456
display(n, a, s)
```

```
#Program to demonstrate Default Argument

def display(name, course = "B.Tech"):
    print("Name is : ", name)
    print("The Course is : ", course)

display(course = "BCA", name = "Ayesh")  # Keyword Argument
display(name = "Kartik")  # Default Argument
```

```
#Program to demonstrate Default Argument

def display(name, course = "B.Tech", marks):
    print("Name is : " + name)
    print("The Course is : ", course)
    print("The Marks is : ", marks)

display(name = "Kartik", 90)  # Default Argument
```

syntax error in your code because the display function call is incomplete

All the arguments to the right of the default arguments must also have defaut values

Corrected Code

```
def display(name,marks, course = "B.Tech" ):
    print("Name is : " + name)
    print("The Course is : ", course)
    print("The Marks is : ", marks)

display(name = "Kartik",marks = 90)  # Default Argument
```

MODULES

Modules are pre-written pieces of code that are used to perform common tasks like generating random numbers, performing mathematical operations etc.

```
import sys
print("\n PYTHON PATH = \n", sys.path)
```

This imports the sys module, which provides system-related information and functionality in Python.

sys.path is a list of directories that Python searches for modules and packages when you import something.
It includes:
The current working directory.
Standard library paths.
Paths of installed third-party packages.

The from.... import statement

The from .. import statement is commonly used in Python to import specific functions, classes, or variables from a module. This allows you to use those specific items directly without needing to prefix them with the module name.

```
from math  import  pi
print(“PI = ”,  pi)
```

```
from math import sqrt as square_root

print(square_root(81))
```

Name of the module

Every module has a name. You can find the name of a module by using the __name__ attribute of the module.

```
print("Hello")
print("Name os this moduel is : ", __name__)
```

Observe the output and always remember that for every program written by user, the name of the module is __main__ .

TASK

1. **Write a program using function to calculate compound interest given the principle, rate of interest and number of years**
2. **Write a function that converts temperature given in celsius in Fahrenhite**

Making your own Module

- You can easily create as many modules as you want.
- Every Python program is a module, i.e., every file that you save .py extension is a module.

STEP 1: Make a file named “MyModule.py”

```
def display():
    print("Hello")

    print("Name of called module :", __name__)
str = "Welcome to the world of python!"
```

STEP 2: Open another file and write your code.

```
import MyModule

print("MyModule str = ", MyModule.str) #using var defined in MyModule
MyModule.display()    #using function defined in MyModule

print("Name of calling module is :", __name__)
```

OUTPUT -

```
MyModule str =  Welcome to the world of python!
Hello
Name of called module : MyModule
Name of calling module is : __main__
```

- **Accessing Module Members: Python allows using the dot operator (.) to access specific members (variables or functions) from a module after importing it.**

- **Selective Import: Instead of importing the entire module, specific members can be imported using the syntax:**

```
from MyModule import str, display
```

- **Module Placement: Modules should be stored in the same directory as the importing program or in a directory listed in sys.path to ensure proper importing.**

- **math Module Overview**: Python's math module provides many standard mathematical functions, including trigonometric functions, logarithmic functions, and constants like pi and e (Epsilon).

- Using **math** Functions: Before using any function from the math module, it must first be imported using import math. Then, you can access its functions or constants using dot notation (e.g., math.pi).

Example Program: A simple Python program using math.pi and math.e is shown:

```
import math
print("pi =", math.pi)
print("Epsilon =", math.e)
```

Output:

```
pi = 3.141592653589793
Epsilon = 2.718281828459045
```

TASK

1. **Write a function that accepts a number n as input and returns the average of number from 1 to n.**
2. **Write a Program to calculate the area of a triangle using function**
3. **Write a program using function to calculate x to the power of y, where y be either negative or positive. Pass dictionary as an argument to the function.**